

Route Clearing

Route Clearing

[Function Description](#)

[Working Principle](#)

[Experiment Steps](#)

[Code Implementation](#)

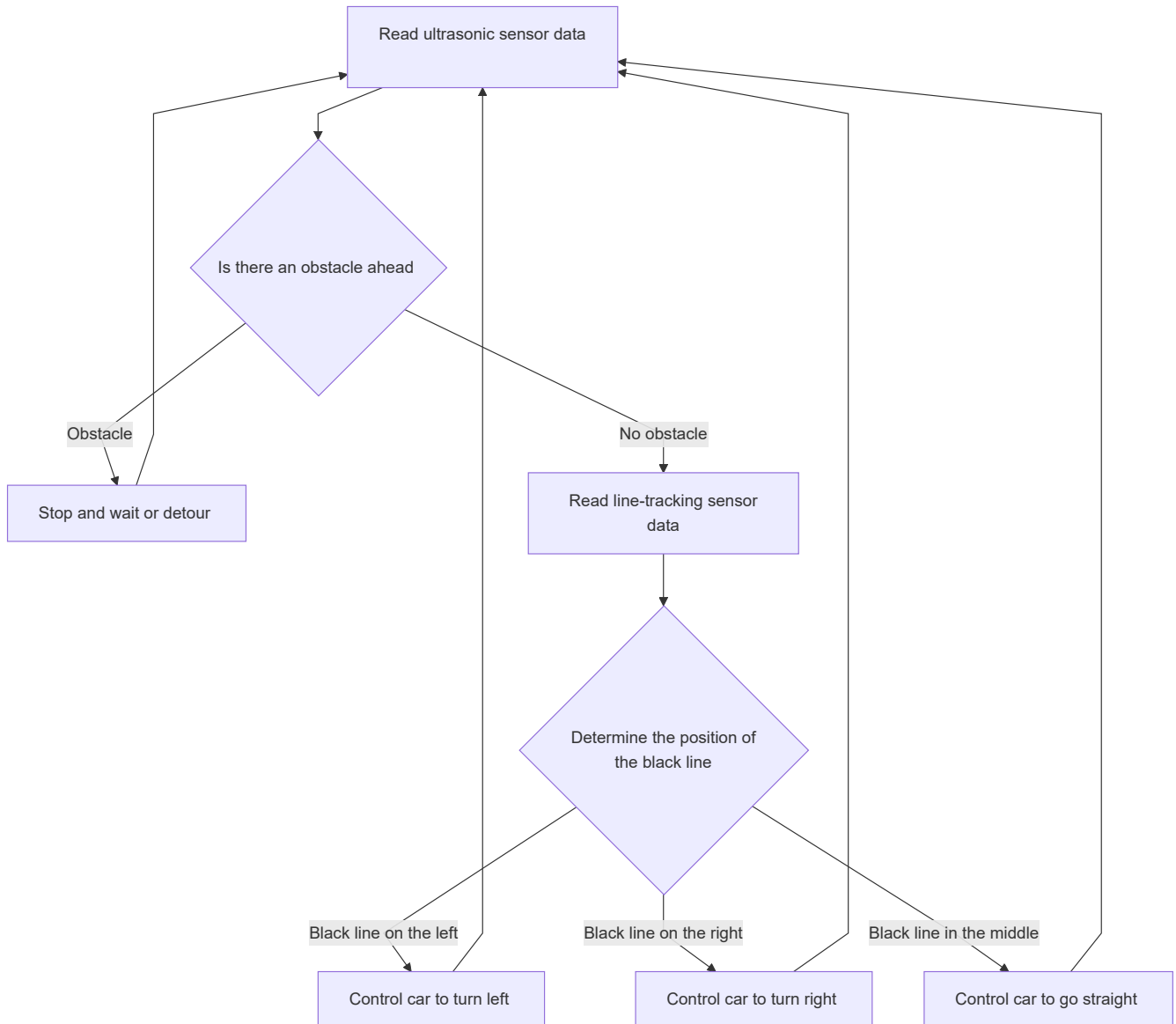
[Experimental Phenomenon](#)

[Notes](#)

Function Description

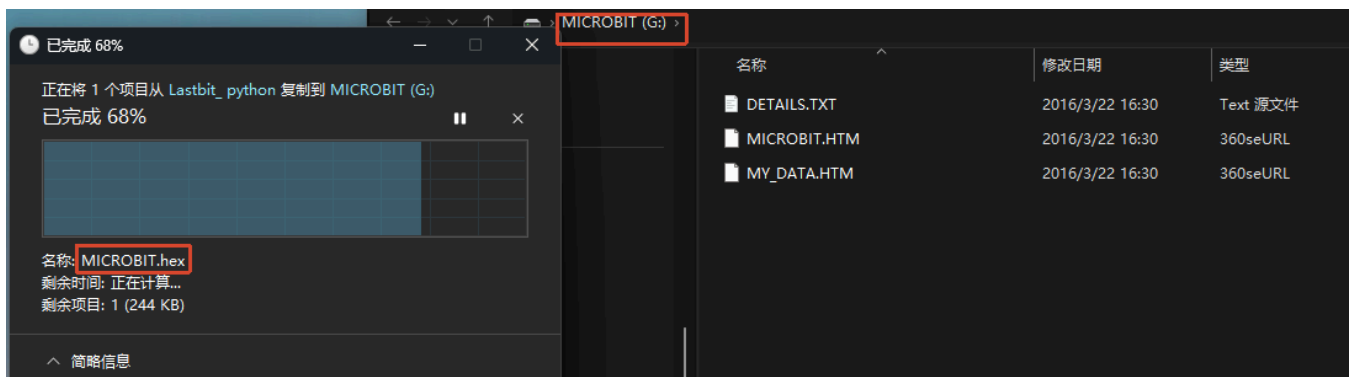
Earlier we learned how to read data from the four-channel line-tracking sensors to implement line tracking. In this section, we will build on line tracking and combine the ultrasonic sensor and servo control to implement the function of line tracking and obstacle clearing.

Working Principle



Experiment Steps

Make sure `LASTBIT.hex` is flashed before flashing, otherwise an error will occur and the module cannot be imported



1. Before flashing, switch the switch to the OFF position and use a microUSB data cable to connect the

computer and the micro:bit main board. **Note that it is the Microbit interface, not the Charging port on the expansion board.**

2. Open the Mu software. Here you can directly copy the program source code we provided; the operation steps are as follows. You can also enter the code manually in the editing window. Note! All English letters and symbols should be entered in English input mode. Use the Tab key for indentation, and the last line should end with a blank program.

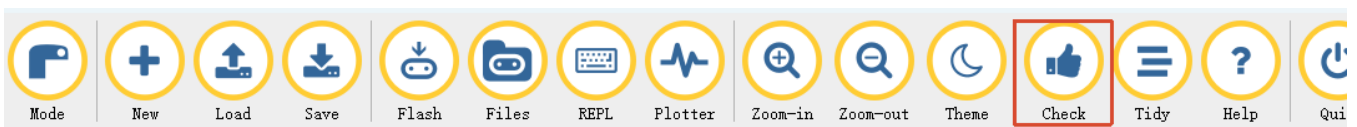
Click the **New** button in the toolbar at the top left, and a file titled "untitled" will appear.



3. Copy the content of the **Code Implementation** section provided below into the current file. You will notice a red dot after the title bar, indicating that the code content has been modified but is in an unsaved state.

Whether to save or not does not affect code checking or flashing.

4. Now you can click **Check** to verify whether there are any problems with the code format.



5. Now if you have already connected the MICROBIT to the PC in advance following the steps above, and flashed **LASTBIT.hex**, the next step is to download the current program to the Microbit.

Click the **Flash** tool in the toolbar to start flashing!

During flashing, the orange-yellow indicator light near the Microbit main board will blink frequently. When the editor shows **Copied code onto micro:bit.** at the bottom, it means the flashing is complete, and the indicator light will no longer blink after flashing.



6. Unplug the microUSB data cable, place the car on the line-tracking track, and switch the switch to the ON position.

Code Implementation

```
# 22. Line Tracking and Obstacle Clearing
from microbit import *
from lastbit import LASTBIT

Lastbit = LASTBIT()

# Speed and proportional coefficient can be adjusted
Kp = 70
base_speed = 80

# Grasp count
grasp_count = 0
# Is grasping
is_grasping = False

# Track data
track = None
# Distance data
distance_mm = 0

# Position bias
pos = 0
# Sensor sum
sum_val = 0
# Last direction
last_direction = 0
# Is lost line
is_lost_line = False
# Lost line start time
lost_start_time = 0

# Initialize servos
Lastbit.set_all_servos(90)
sleep(1000)
Lastbit.set_servo('S2', 150)
sleep(1000)

display.show(Image.HAPPY)

def tracking_mode():
    """Tracking mode"""
    global track, sum_val, pos, last_direction
    global is_lost_line, lost_start_time

    if track is None:
        return

    # Data conversion: 0 to 1, 1 to 0
    left = 1 if track[0] == 0 else 0
```

```

mid_l = 1 if track[1] == 0 else 0
mid_r = 1 if track[2] == 0 else 0
right = 1 if track[3] == 0 else 0

sum_val = left + mid_l + mid_r + right

if sum_val > 0:
    is_lost_line = False
    lost_start_time = 0

    # All sensors detect line (crossroad)
    if sum_val == 4:
        Lastbit.car_control('FORWARD', base_speed, 100)
        return

    # Calculate position bias
    pos = (left * -2 + mid_l * -1 + mid_r * 1 + right * 2) / sum_val

    if pos < -0.5:
        last_direction = 1
    elif pos > 0.5:
        last_direction = -1

    # Proportional control
    turn = Kp * pos
    left_speed = base_speed + turn
    right_speed = base_speed - turn
    Lastbit.set_motor(int(left_speed), int(right_speed))
    return

# Lost line handling
if not is_lost_line:
    is_lost_line = True
    lost_start_time = running_time()

lost_time = running_time() - lost_start_time
if lost_time > 1000:
    Lastbit.stop_motor()
    if last_direction == -1:
        Lastbit.car_control('SPINRIGHT', 160, 200)
    elif last_direction == 1:
        Lastbit.car_control('SPINLEFT', 160, 200)

# Grab action
def do_grasp():
    global is_grasping, grasp_count, distance_mm

    is_grasping = True
    Lastbit.stop_motor()
    distance_mm = Lastbit.ultrasonic_distance()

    # Move backward if too close

```

```
if distance_mm < 68 and distance_mm >= 10:
    Lastbit.car_control('BACKWARD', 20)
    sleep(50)
```

```
# Lower to prepare
```

```
Lastbit.set_servo('S2', 80)
Lastbit.set_servo('S3', 90)
Lastbit.set_servo('S1', 90)
sleep(200)
```

```
# Close claw
```

```
Lastbit.set_servo('S1', 160)
sleep(200)
```

```
# Lift up
```

```
Lastbit.set_servo('S2', 160)
sleep(200)
```

```
# Rotate
```

```
Lastbit.set_servo('S3', 155)
sleep(200)
```

```
# Lower down
```

```
Lastbit.set_servo('S2', 80)
sleep(200)
```

```
# Open claw
```

```
Lastbit.set_servo('S1', 90)
sleep(200)
```

```
# Reset
```

```
Lastbit.set_servo('S2', 160)
sleep(200)
```

```
Lastbit.set_servo('S3', 90)
sleep(1000)
```

```
is_grasping = False
```

```
grasp_count += 1
```

```
while True:
```

```
    track = Lastbit.read_track()
```

```
    distance_mm = Lastbit.ultrasonic_distance()
```

```
# Move backward if too close
```

```
if distance_mm < 70 and distance_mm >= 10:
    Lastbit.car_control('BACKWARD', 20)
    sleep(50)
    continue
```

```
# Execute grab when obstacle detected and not grasping
```

```
if (
```

```
    not is_grasping and
    distance_mm <= 90 and
    distance_mm >= 70 and
    grasp_count < 2
):
    do_grasp()
    continue

# Reset grasp count
if grasp_count >= 2:
    grasp_count = 0

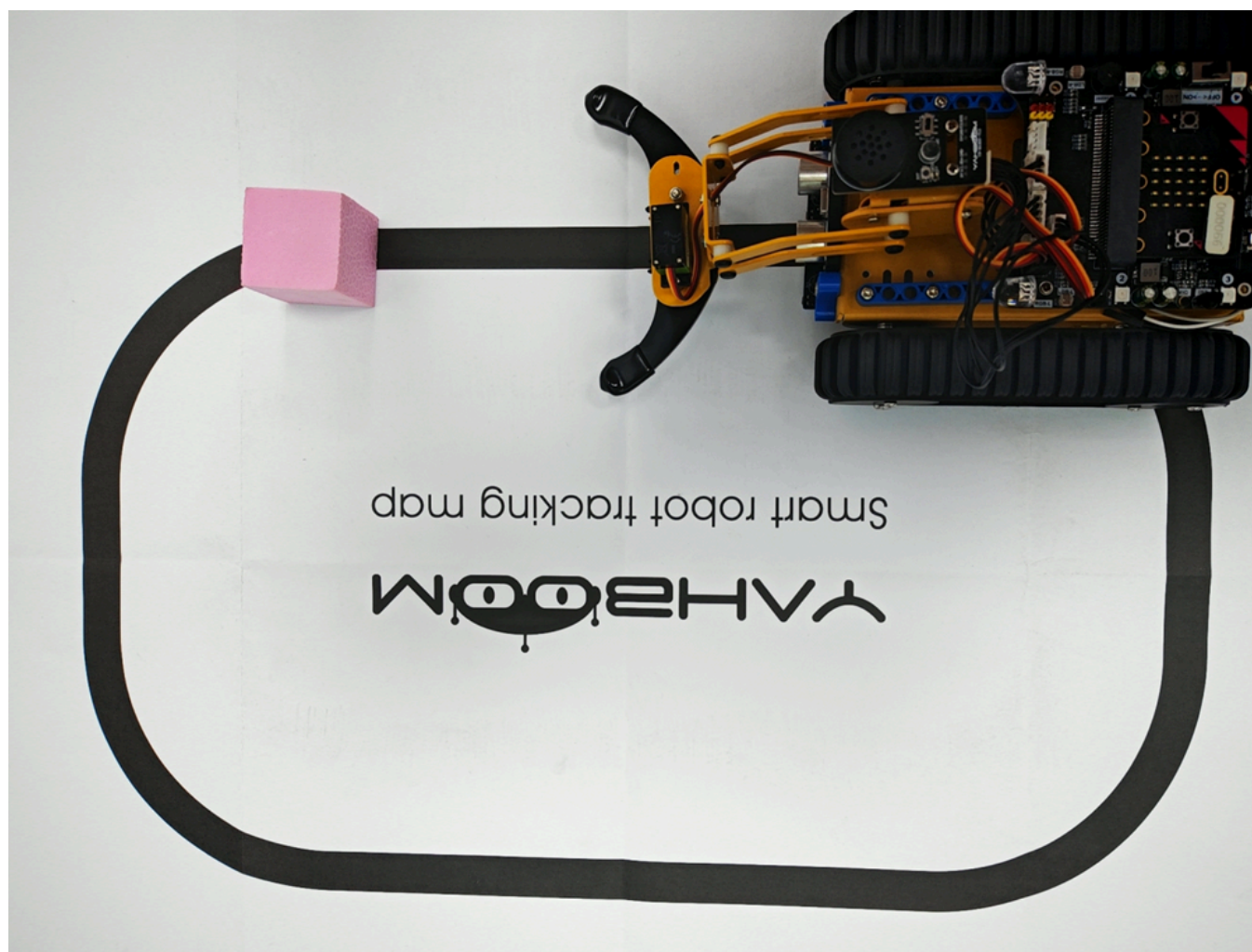
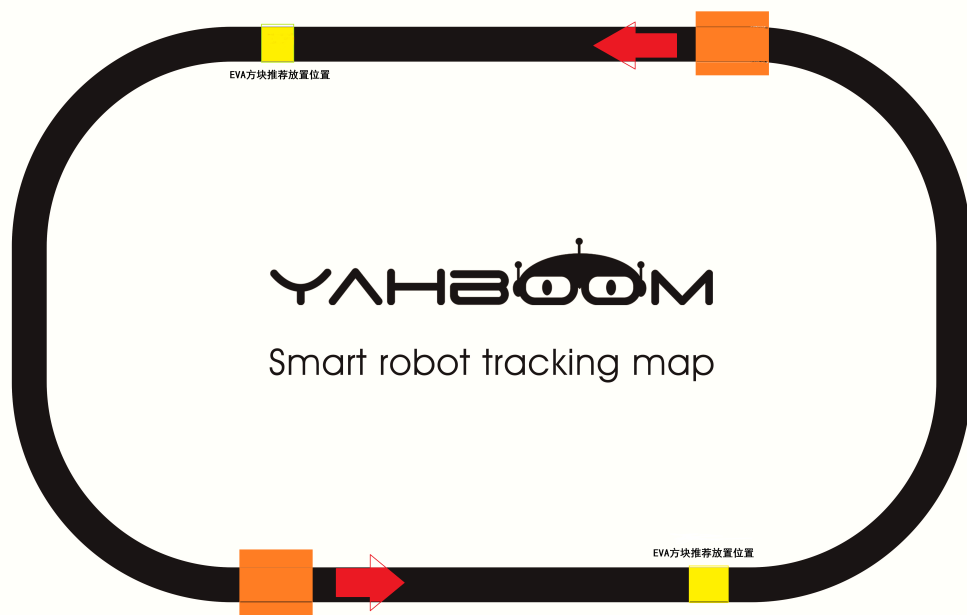
# Execute tracking when not grasping
if not is_grasping:
    tracking_mode()

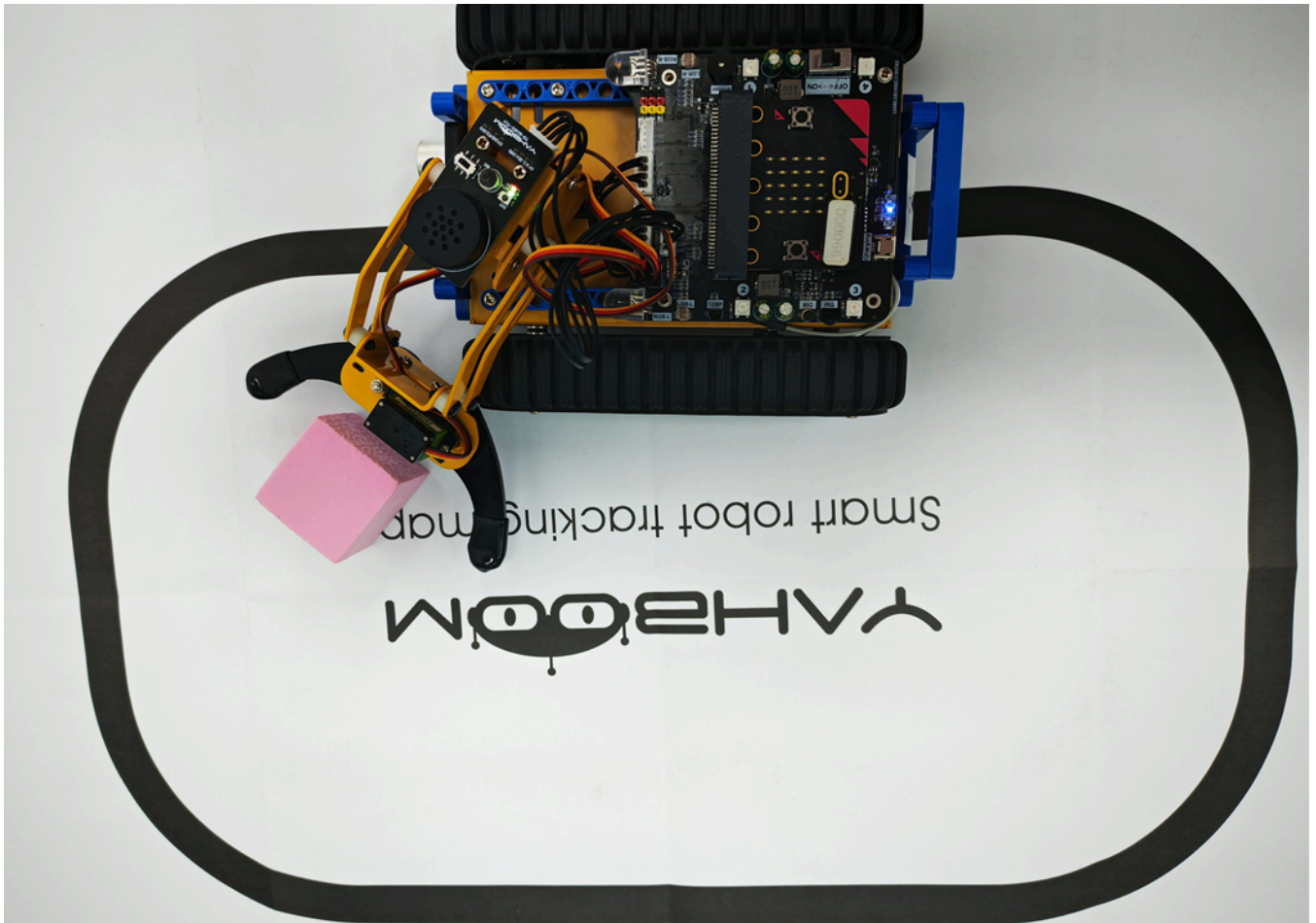
sleep(50)
```

Experimental Phenomenon

If the battery level is low, you can adjust the straight and turn amplitudes by modifying base_speed and Kp respectively. The larger base_speed is, the faster it goes straight; the larger Kp is, the greater the turn amplitude.

The car will track the line according to the direction of the track. When the ultrasonic sensor detects that the distance to the EVA block reaches the grab range, the car will stop and clear the block. Place the block at the left front of the car (when the obstacle distance is too close, the car will back up to adjust the distance). After the ultrasonic sensor detects no obstacle, it continues to track the line. If you need to adjust the grab distance range of the car, you can modify the relevant parameters in the program. It is recommended to place the EVA obstacle block on the second half of the straight section after the car turns, where the ultrasonic distance measurement is more accurate.





Notes

- The sensitivity of the line-tracking sensor needs to be adjusted to the appropriate position via the potentiometer
- Make sure the battery is fully charged
- It is recommended to place the EVA obstacle block on the second half of the straight section after the car turns, where the ultrasonic distance measurement is more accurate